Arquitectura de Computadoras

Jaime Nazar Anchorena

Contents

Unidad 1: Codigo en bajo nivel **3**
Programas y binarios . . . 3
Compilacino y linkedicion de C(de PI) . . . 3
Ejecucion del programa . . . 3
Representacion en memoria . . . 3
Programa en disco . . . 3
Analisis de binario(linux) . . . 5
`file` . . . 5
`strings` . . . 5
Bless . . . 5
Llamadas a funcion(call) . . . 5
Llamadas al sistema operativo(System Calls) . . . 6
Formas de ejecutar System Calls . . . 6
Procesadores y Lenguaje ASM(en Intel) . . . 7
Registros de Intel para programas . . . 7
Uso del dissasembler . . . 7
Assembler de Intel . . . 8
Instrucciones . . . 8
Modos de direccionamiento . . . 10
Registros de flags . . . 10
Ejemplo . . . 10
Compilacion y ejecucion . . . 11
Argumento por linea de comando . . . 11
Syscalls . . . 12
Assembler y C . . . 12
Instruccion `RET` . . . 12
Pasaje de parametros . . . 13
Llamado de funciones . . . 13
Convenciones en C . . . 13
Llamado de C a ASM . . . 14
Compilacion . . . 15
Deteccion stack smashing . . . 15

Unidad 2: Transmision digital **15**
Sistema de entrada y salida 16

Unidad 1: Codigo en bajo nivel

Programas y binarios

Compilacino y linkedicion de C(de PI)

Para un lenguaje se convierta en ejecutable pasa por una serie de pasos, en uno de los pasos el `.c` se conviernte en un `.s` (Assembler) y ese es el que nos interesa. `gcc` primero invoca el proprocesador `cpp`, el cual toma el coodigo fuente original y realiza la macroexpansion de diretivas como `#include` y `#define`.

Luego `gcc` toma el control de esa salida y convierte el código en C en código equivalente en Assembler. Genera un archivo con extensión `.S`. Este código en ASM puede ser generado en sintaxis AT&T ó en la sintaxis Intel. Luego el compilador GNU de Assembler "gas" se encarga de generar el codigo objecto `.o`. En Linux el ejecutable podrá ser de diferentes formatos, por ej, ELF (mas reciente) y `A.OUT` .Se debe tener en cuenta en este punto que el linkeditor `ld` también puede generar el modelo flat, también llamado binario.

Programa `.c` -> Preprocesador(cpp)(`.c`)) -> gcc (`.s`) -> gac `.o` -> Id Ejecutable

Ejecucion del programa

Linea de comando -> Sistema Operativo -> Lectura del disco -> Asignacion de espacio en memoria -> `CALL` o `JMP` a direccion de memoria de programa -> Ejecucion del programa

Representacion en memoria

Los programas antes de ser ejeutados van del disco a la memoria, esto permite ejecutarlos mas de una vez sin tener que volver a cargarlos y hace que la lectura y ejecucion del programa sea mas rapida.

- **Codigo**: Instrucciones del programa
- **Datos**: Variables estaticas y globales que si inicial al cargar el programa
- **Heap**: Memoria dinamica que se reseva y se libera en tiempo de ejecucion
- **Pila**: Argumentos y variables locales

Para que corra el programa, el sistema operativo debe dejar punteros a las distintas secciones del programa, mas especificamente: un puntero al codigo, punteros a los datos, y puntero a la pila.

Programa en disco

La primer parte se conoce homo `header` o encabezado, tiene informacion para el sistema operativo, por lo tanto, depende del sistema operativo. La segunda parte es el codigo en si.

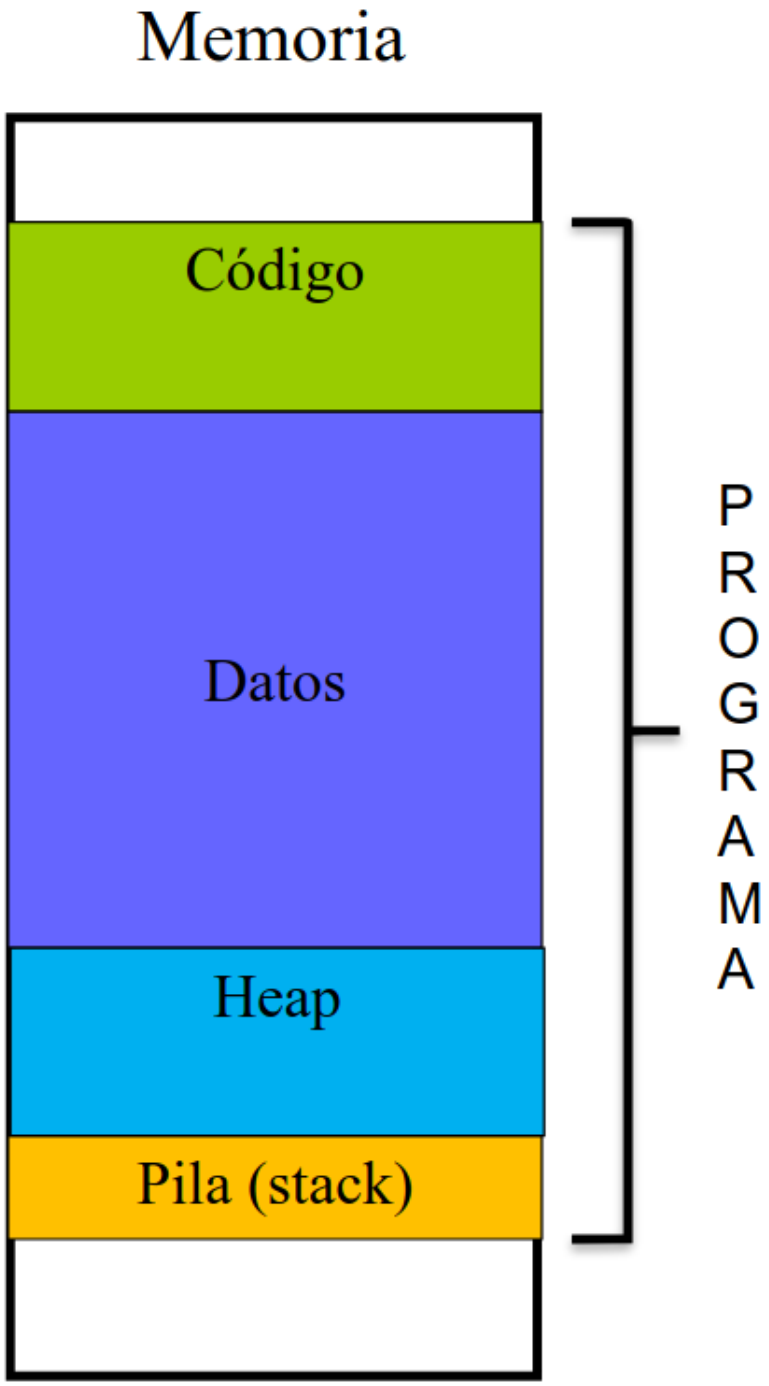

Figure 1: Representacion de un programa en memoria

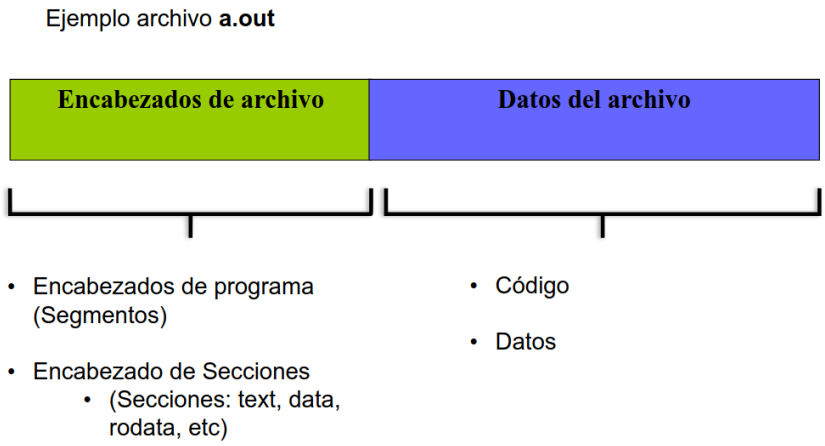

Figure 2: Representacion programa en memoria

Analisis de binario(linux)

`file`

Se puede ver que tipo de archivos es con el comando `file`, podria ser un `ELF`(Executable Linux Format)

`strings`

Es un programa que permite extraer todos los `string` que se encuentran dentro del executable en la seccion de datos. Sin embargo, no lo puedo modificar directamente, para ello conviene usar un editor hexadecimal.

Bless

Permite leer y editar archivos en su forma mas basica, hexadecimal(binario).

Llamadas a funcion(call)

La isntruccion `CALL` de Intel permite el llamado a una funcion, guarda en la pila la direccion de retorno. La funcion debe terminar con la instrucion `RET` de Assembler, la cual saca el puntero a la promixa intruccion del stack y el programa continua desde ahi.

```
main:
...
CALL func 1
...
func1:
```

```
...
RET
```

Toda llamada a funcion se resuelve con un `CALL`

Llamadas al sistema operativo(System Calls)

El sistema operativo se encarga de asignar memoria, verifica quien tiene permiso y quien no, dependiendo de los permisos de cada programa, estos podram hacer determinadas llamadas al sistema operativo.

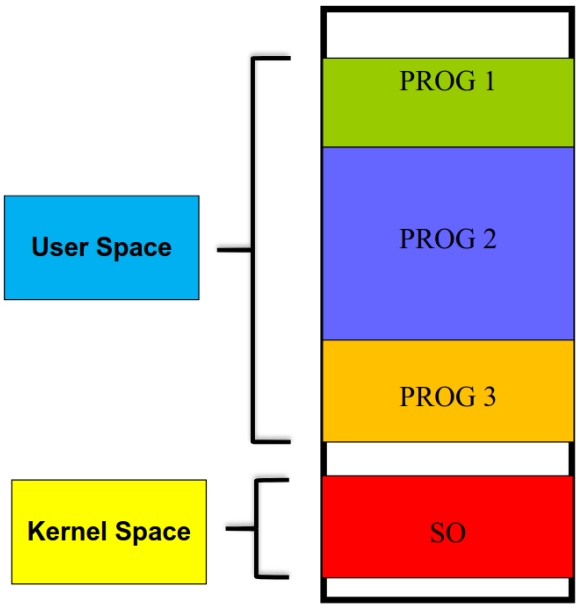

Figure 3: Representacion de memorio del sistema operativo

User space (system call)-> Kernel space

En la documentacion, estan las declaraciones de System Calls, donde se especifica los parametros y que hace cada una. Estas instrucciones reciben los parametros a traves de los registros(siempre por los registros).

Ejemplo: Para imprimir en pantalla, el system call que utiliza linux es `write(1, "wawa", 10wawa)`

Formas de ejecutar System Calls

- `INT 80h`: Interrupcion numero 80, consume mas tiempo
- Instrucciones `SYSCALL/SYSRET` (Intel) y `SYSenter/SYSEXIT` (AMD): Disponibles desde Pentium II, la libreria de C depende de la arquitectura. Sirve para saber donde estoy en relacion al programa.

- `vsycall` y `VDO`: Syscall virtuales y Virtual Dynamic Shared Object, linux crea paginas de moria en user space para acelerar tiempos.

Procesadores y Lenguaje ASM(en Intel)

Registros de Intel para programas

- `IP`: Instruction Pointer(`EIP` en 32 bits y `RIP` en 64 bits)
- `SP`: Stack Pointer(`ESP` en 32 bits y `RSP` en 64 bits), apunta al final de la fila.
- Registros de manejo de memoria:
  - `CS`: Code Segment, apunta al inicio del segmento en memoria(posicion absoluta)
  - `DS`: Data Segment(Tambien `ES`, `FS` y `GS`), la direccion de la informacion desde el inicio del data(posicion relativa). Para obtener la relacion absoluta, se le suma el `CS` al `DS`.
  - `SS`: Stack Segment, apunta al inicio de la fila.

  Nota: Los registros de segmento mantienen su ttamano en arquitectura de 32 y 64 bits.

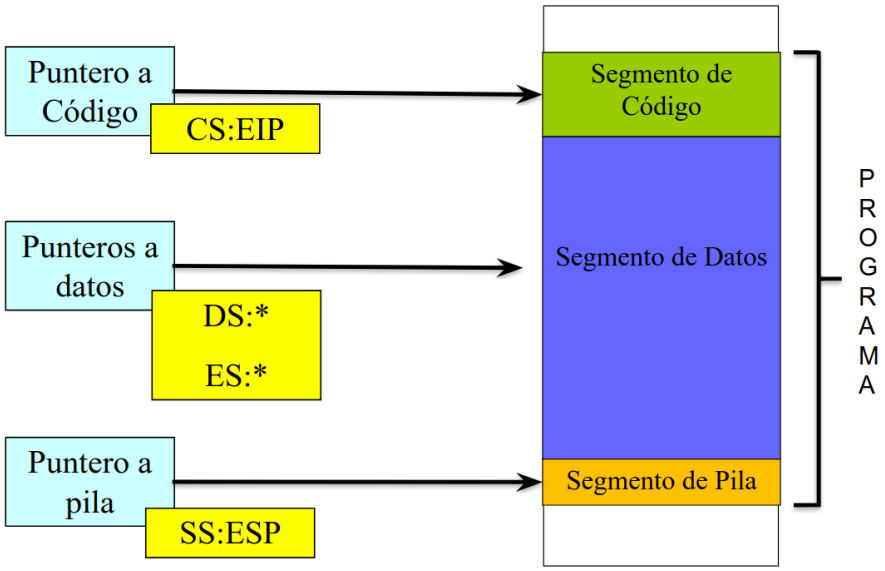

Figure 4: Representacoin de los registros en la ejecucion de un programa

Uso del dissasembler

Basicamente tenemos que pedirle que lo decompile en el Assembler de Intel pues ese es el que vamos a utilizar en la materia.

`objdump -d -M intel [nombre del programa]`

> **Nota**: Siempre se puede usar `man` para ver que hace cada cosa.

Assembler de Intel

El Intel 80386 trajo un cambio de paradigma introdujendo varias features;

- Multitarea
- Multiusuario
- Tiempo compartido
- Tiempo real
- Sistema de proteccion

Lo importante de todo esto es que son **features** de hardware, el procesador las tendra independientemente del software que esta corriendo. Este procesador tendra registros de 32 bits y sosportte de hasta 4Gb de RAM.

Intel mantubo la retrocompatibilidad, con 8086 por ejemplo, manteniendo los registros `AH` y `AL`, que forman parte del `AX`(De 16 bits), el cual a su vez es mitad del `EAX`.

Hay dos tipos de Assembler principales, el de AT&T y el de Intel, nosotros vamos a usar esclusivamente el de Intel.

Instrucciones

Ordenes que se le dan al procesador, por ejemplo:

```
add eax, 0x1
```

Para saber que hace cada instruccion tenemos la cartilla. Las instrucciones en Intel tienen tamano variable, si mencionas un valor numerico y no poner la `h` al final, se toma como un numero decimal. Ademas, todas las direcciones de memoria tienen un byte, si pido 16 bits de `109h`, entonce va a traee `109h` y `10Ah`.

> **Nota**: Si intentas usar una instruccion invalida, da error en tiempo de compilacion. Por ejemplo, `mov AX, 000099h` da error pues los `0` los cuenta.

Orden ejecucion: Busca direccoin de memoria -> Decodifica instruccion -> Si se modifica un registro lo hace, y si tiene que modificar memoria tiene que volver a memoria.

Para saber cuando bytes tiene que buscar o cualquier otra opcion de la misma instruccion, existe el micro-codigo, el cual es interno del procesador y contiene todas las opciones, por ejemplo, `movw`, `movb`, etc. Estas se pueden entender como un switch case interno del procesador.

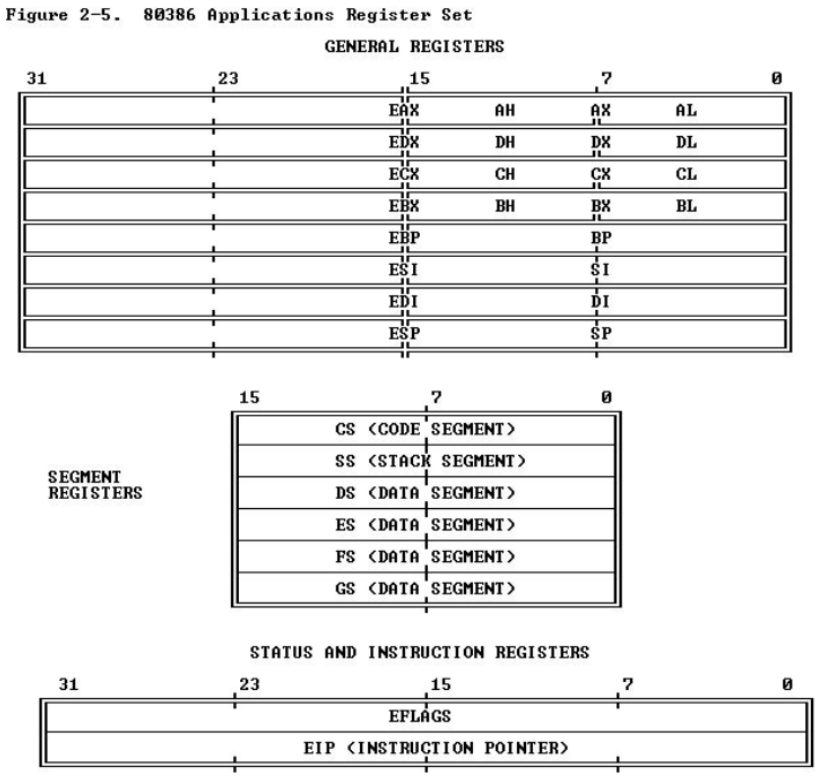

Figure 5: Registros del Intel 80886

Modos de direccionamiento

- Direccionamiento inmediato: `mov ax, 0fffh`
- Direccionamiento de registro: `mov eax, edx`
- Direccionamiento directo o absoluto: `mov ax, [0564h]`
- Direccionamiento indirecto: `mov cx, [bp]`
- Direccionamiento como indice o indexado: `mov cd, [bp+4]`

  Nota: NO existe `mov [bx], [ax]`, si quiero hacer eso necesito usar un registro de intermediario.

Registros de flags

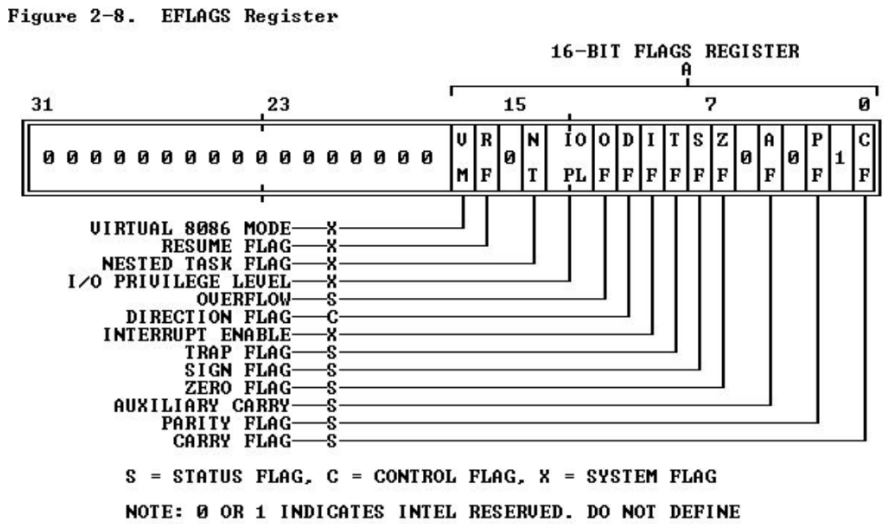

Figure 6: Registros de flags

Ejemplo

```
section .text
    global _start

_start:

    mov dx,0FFh
    mov bx,20h
    add dx,bx
    push dx
    push 4
```

```
    pop cx

Ciclo:
    inc bx
    dec cx
    jnz Ciclo

    mov eax,parametros

    mov AH,[parametros]
    mov BL,[parametros+1]
    add ah,bl
    mov [salida],ah

    ret 0

section .data
parametros db 11h,12h,13h
salida db 0
```

Instrucciones como `section .text` son especificas del compilador, en nuestro caso `nasm`, y no son las mismas para todos los compiladores. Por ejemplo, el `#include<>` de C es especifico para el comilador y no lenguaje C en si.

Compilacion y ejecucion

Para compilar:

```
nasm -f elf64 ejemplo.asm -o ejemplo.o
```

Para linkeditar:

```
ld ejemplo.o -o ejemplo
```

O `gcc` tambien sirve para linkeditar. `ld` lo pasa directo a ejecutable y nada mas, mientras que `gcc` le agrega mas informacion, le agrega headers. El gcc sirve mucho para likeditar assembler con C, mientras tanto `ld` seria la opcion mas "pura".

Esto genera el ejecutabl `ejemplo`.

> **Nota**: El flag 0 lo puede levantar cualquier instruccion en cualquier momento

Argumento por linea de comando

Para destacar, el primer argumento es siempre el nombre del programa que se esta corriendo y el ultimo es `NULL`. Esto es independiente del lenguaje y es parte del funcionamiento del sistema operativo. En un sistema Unix, el sistema

operativo, antes de darle control al programa configura el stack para que tenga los argumentos del programa. El Stack al inicio del programa queda:

Dir.	Data
`ESP`	Cantidad de argumentos
`ESP+4`	Path al programa
`ESP+8`	Direccion del 1er arg.
(...)	(...)
	`NULL`

Pero no es practico usar `ESP`, por ello se arma un **stack fram**, es decir, se guarda el valor de `ESP` en `EBP`.

Syscalls

Vamos a usar las syscalls de linux, sino habria que buscar en la doc cual es el codigo de lada syscall. En `EAX` pones el numero de la syscall que queres ejecutar y en `EDX` pones el numero de retorno. Hay syscall con argumentos variables y otras con argumentos tanto obligatorios como opcionales(depende del sistema operativo).

Una syscall es una funcion del sistema operativo, dependen del sistema operativo y permiten interactuar con el sistema. Estan ubicadas en el sistema operativo mismo, es decir que en tiempo de ejecucion estan en la memoria dedicada al kernel space, son parte del Kernel.

Ejemplo: Llamada a syscall print.

```
mov ebx, 1
mov eax, 4
int 80h     ; Llama a print
```

Assembler y C

Vamos a mezclar assembler con C, donde vamos a poder tener funciones y variables del lado de C, y funciones y variables del lado de assembler. Para que sean compatible entre los dos, se va a usar la pila y los registros. Desde assembler nos vamos a adecuar con lo que hace C, ya que C es fuertemente tipado y eso va a hacer la interaccion mas facil.

En un procesador Intel, la pila comienza en el tope(`0x....FFFFh`) y decrece a nivel puntero en la memoria, el `ESP` sera el **stack pointer**.

Instruccion `RET`

Cuando se ejecuto, el procesador toma el contenido de los apuntado por `ESP` y salta a esa posicion de memoria. Es equivalente a hacer un `JMP` a esa direccion.

Es muy importante no cambiar el `esp` mientras se ejecuta una funcion, o al menos dejarlo en el mismo lugar que lo econtro. Entonces es muy importante para nosotros no perder la direccion de memoria de retorno, entonces por seguridad nos cenvendria hacer una copia. Eso es lo que hace C detras nuestro, se encarga de mantener una capia y antes del return deja el stack tal y como estaba.

Si esto falla ocurre el famoso `Segmentation Fault`, hay que tener mucho cuidado porque en assembler no hay mucha proteccion.

Pasaje de parametros

```
int funcion(tipo1 par1, ripo2, par2) { ... }
```

El pasaje depende de la qrquitectura del compilador:

- **32 bit**: Todos se pasan por la pila. Esto genero problemas de seguridad y lo cambiaron(En memoria tambien hay codigo).
- **64 bit**: Se pasan primero por registros, y si no alcanzan(hay una cantidad finita de registros), luego se pasan por la pila.

En assembler no hay una forma determinada.

Para pasar argumentos por registro se usan `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`(Si tengo un procesador de punto flotante hay mas, pero no los vamos a ver). Si la funcion necesita mas parametros se usa la pila. Todo esto es importante para mezclar con assembler, necesitamos entender como funciona el compilador. Los registros son especificos del C, no depende del hardware.

Reglas:

- Hay registros que esta prohibido usar.
- Segun el tipo de dato se usan diferentes registros.
- Si los argumentos no entran en los registros van a la pila.

Llamado de funciones

Las funciones se llaman con la pila, esto juego un rol fundamental en arquitectura x86. En este espacio de memoria se almacenan las variables locales de la funcion llamadoa, sus argumentos y su direccoin de retorno. Aparece el concepto de **frame**, es la parte de la pila que le corresponde a cada funcion. Se crea en tiempo de ejecucion.

Convenciones en C

Los parametros se pushean en el stack de derecha a izquierda, para backupear `ESP` lo vamos a guardar en otro registro, en `EBP`(base pointer). Antes de asignarle `ESP` a el `EBP`, se pushea el `EBP` al stack para restaurarlo despues. Esto se conoce como el armado del **stack frame**.

```
push ebp
mov ebp, esp
```

Luego para desarmar el stack es la operacion opuesta.

```
mov esp, ebp
pop ebp
```

Estas son las operaciones que realiza una funcoin en C para asegurarse de no perder el `ESP`. Para retornar un valor, si el valor es menor a 32 bits se retorna el `EAX`, si es mayor retorna la parte alta en `EDX` y la parte baja en `EAX`. Si es mas complejo (ej. Estructura de datos) retorna un puntero formado por `EDX:EAX`. En el decompilado de C existe una macro llamada `leave`, esta realiza el desarmado del stack frame.

> **Nota**: Se llama base donde empieza la pila.

Llamado de C a ASM

En C utilizamos:

```c
#include<stdio.h>

extern unsigned int siete(void);

main() ....
```

Notar la palabra clave `extern`, luego en al assembler que luego voy a linkeditar con la parte de C:

```
...
siete:
    ...
```

Si no armo el stack frame, no va a pasar nada, pero es mucho mas prolijo y buena practica hacer.

> **Nota**: Numca usar `__asm__("movl $ $0x12345, %eax");`, se conoce como inline asm en C(Al profesor le parece un horror).

Tenemos dos formas de ver el codigo C convertido en ASM:

- Compilar con la flag `-S`
- Usando GDB

Va a haber operaciones que podrian no tener sentido, pero para que el compilador pueda compilar la infinidad de combinaciones diferentes de programas hay veces que lo que resulve dan ese tipo de cosas. Lo que ocurre es que el compilador va de a bloques y siempre avanza, no lo vuelve a revisar.

> **Nota**: `mov DWORD PTR[esp], 0x3` especifica que queres que el numero 3 sea guardado en 4 bytes(a pesar de que ocupa 2 bits como

minimo).

Compilacion

Para la mayoria de los casos la compilacion se hace en una sola pasada, eso causa que al analizar codigo compilado aparescan operaciones que pueden no tener sentido, aunque no afectan el funcionamiento del codigo. Por ejemplo, reservar espacio de mas en la pila. Sin embargo, hay casos en los que si se hacen mas de una pasado, por ejemplo, los sistemas operativos, los cuales tienen millones de lineas de codigo y necesitan toda la optimizacion que puedan conseguir.

Deteccion stack smashing

Lo que hace `gcc` cuando no agregas ninguna flag, pone en el stack lo que se conoce como `canary`. Si al ejecutarse se detecta que cambio alguno de los `canary` entonces interrumpe la ejecucion pues detecto un `stack smashing`(aborta ejecucion).

Lo pone justo despues de armar el stack frame, el valor es un valor random que saca de una seccion de memoria. Luego, antes de que termine el programa llama a una funcion llamada `__stack_chk_fail` para verificar si el `canary` siga igual.

Unidad 2: Transmision digital

Vamos a analizar como se comunica el procesador con el resto del hardware. Para representar 0s y 1s en un cable de cobre hay varias maneras, una manera es voltaje o no voltaje, otra seria +3,3V o -3,3V. El procesador se comunicara de esa manera, hay dos maneras:

- **Transmision serie**: Va todo por el mismo tiempo, cada bit ocupa un paso.
- **Transmision paralela**: Si quiero transmitir 8 bits en un solo paso, es la manera mas rapida y la que se usa en la computacion, se conoce como un **bus**.

En las computadores y circuitos chicos tenemos los bus, pero en las redes que conectan al mundo y usan miles de kilometros de cable se utiliza la transmision en serie a pesar de ser mas lente(debido al costo principalmente).

Hubo dos propuestas principales para como estructurar la conexino del CPU con memomria:

- **Von Neumann**: Propone un solo bus entre la memoria(codigo y datos) y el CPU
- **Harvard**: Propone dos buses, uno a la memoria con los datos y otro para la memoria(otra memoria) con el codigo. De esa manera de paraleliza la lectura.

Sin embargo, hoyy en dia se utiliza Von Neumann a pesar de ser mas lento debido al costo y complejidad reducida.

Sistema de entrada y salida

Tenemos dos buses principales, un bus de **direcciones**(address) y otro de **datos**. Estan todos los perifericos conectados a los mismos cables, para que no se crucen y se comuniquen todos de manera ordenada se utiliza el bus de direcciones, este se encarga de apuntar a la direccion de memoria que quiero escribir o leer.